

Refund Desk

An internal tool for a support team to log, review and resolve customer refund requests.

SOURCE AND DEPLOYMENT

REPOSITORY

github.com/davncntcode/refund-desk

LIVE DEPLOYMENT

app-refunddesk.vercel.app

RUNNING IT

Four commands, no services to install

You need **Node 20.12 or newer** (`.nvmrc` pins 22) and npm. No Docker, no Postgres, no cloud account, no API keys.

```
git clone https://github.com/davncntcode/refund-desk.git
cd refund-desk
npm install
npm run db:migrate # creates data/refunds.db, applies drizzle/
npm run db:seed    # optional: 44 demo requests
npm run dev        # http://localhost:3000
```

No environment file is needed for local work. `DATABASE_URL` falls back to `file:./data/refunds.db` in both the app and the migration config. Copy `.env.example` to `.env.local` only to point somewhere else, such as a shared Turso database. The seed is deterministic and clears both tables first, so it is safe to re-run whenever you want a clean slate.

CHECKS

Everything the build enforces

```
npm run lint      # eslint
npm run typecheck # route types, then tsc --noEmit
npm run build     # production build
npm test         # 63 unit tests
npx playwright install chromium # once
npm run e2e       # 3 end-to-end tests
```

The end-to-end run builds the app and points it at its own database (`data/e2e.db`), wiped and migrated before the server starts, so it never touches your working data.

NOTE · 01

Assumptions I made

- **It sits behind something.** Scoped as an internal tool on a network or SSO that already authenticates the team, so it has no login of its own. That one assumption drives the biggest limitation below.
- **The brief named a status field but not the statuses.** The five-state machine, and the rule that a rejection needs a reason, are my design. A team may want different states, so they live in one file.
- **One currency, one timezone** — USD and UTC.
- **Reason is a category plus a description.** The brief asked for "reason"; a free-text box alone cannot be filtered or reported on, so a request carries both.
- **Hundreds to thousands of requests, not millions.** Offset pagination, a 5,000-row export cap and `max(reference) + 1` are comfortable at that size and would not be at ten million.
- **The amount is what the customer asked for**, not what was eventually paid.
- **Whoever runs this next has no cloud accounts**, so the default database is a local file.

NOTE · 02

Improvements beyond the brief

- The status workflow is **enforced in the Server Action**, not only in the UI, so a direct POST cannot skip review or reopen a closed request
- An **audit timeline** per request, written in the same transaction as the status change
- A **dashboard** — open exposure, queue standing, 14-day intake, oldest waiting
- **Search, five sort orders and pagination**, all in the URL so any view is shareable
- **CSV export** of the active filter, hardened against spreadsheet formula injection
- **Human references** (RF-2026-0001) rather than exposing ids
- **Light and dark themes**, designed empty states, and a card list rather than a sideways scrolling table on phones
- **66 tests** — 63 unit, 3 end-to-end against a real build on its own database
- A **deterministic seed**, contrast **verified by computation** in both themes, and money as **integer cents** end to end

NOTE · 03

Known limitations

- **No authentication, so no actor on the audit trail.** Changes record what and when, not who. Server Actions are reachable by direct POST and carry no authorisation check today.
- **libSQL/SQLite takes one writer at a time.** Fine for a support team, not for high write concurrency. Reference generation would need a real sequence under heavy concurrent creates.
- **Offset pagination** slows down deep into a very large table, and **export is synchronous** and capped at 5,000 rows.
- **Average resolution** is `updated_at - created_at`, which would skew if a row could be edited after it was resolved. Today it cannot.
- No partial refunds, no currency beyond USD, no attachments, no customer notifications. No rate limiting, and no CI pipeline.
- Accessibility was verified by computation and code review — contrast, focus rings, labels, semantics — but **not with an actual screen reader**.

What I would do next with more time

- 1 Authentication with agent and manager roles**, and an actor on every status event. It closes the one real security gap and unblocks everything else here.
- 2 CI on every pull request** — lint, typecheck, unit and e2e — plus a deploy preview.
- 3 Tell the customer.** The tool records what the team decided; the email is still a human step.
- 4 Connect the money.** "Refunded" is a status, not a payment. Wiring it to the payment provider makes the record true rather than asserted.
- 5 Ageing and SLA alerts** on the pending queue, since "waiting longest" is the screen a team would live in.
- 6 Bulk actions** on the list, plus keyset pagination and a streamed export if volume grows.
- 7 A screen reader pass**, to confirm what the computed checks only imply.

FOR REFERENCE

What it is built on

Framework	Next.js 16, App Router, TypeScript
UI	Tailwind CSS v4 and shadcn/ui, on a defined visual direction
Forms	react-hook-form and zod, one schema shared by the form and the action
Data	Drizzle ORM over libSQL — a local SQLite file, or Turso in production
Writes	Server Actions only, re-validated with zod at the boundary
Tests	vitest for the rules, Playwright for the flow
Design	<code>docs/DESIGN-BRIEF.md</code> and <code>docs/DESIGN-SYSTEM.md</code>